



LUẬN VĂN TỐT NGHIỆP
C04337 | HK252

Ứng dụng DevOps và container hóa trên Kubernetes cho hệ thống Moodle LMS tích hợp AI

GVHD: TS. Nguyễn Quang Hùng

SVTH1: Trần Phước Nhật - 2212412

SVTH2: Nguyễn Đăng Cường - 2210432

Ho Chi Minh, 10/06/2026

Table of CONTENTS

- 1 Overview & problem identification
- 2 Analysis, Design & Implementation
- 3 Results and future development
- 4 Demo



Overview & problem identification

Overview

- **Context:** Digital transformation trends in education
- **Need:** A flexible and highly scalable LMS system
- **Solution:** Moodle on Kubernetes with integrated AI
- **Technologies:** Cloud-native, containerization, and automation

Welcome to the Moodle community

The place to get support, ask and answer questions and contribute to the open source learning platform, Moodle LMS.

[Get involved →](#)



54,140,810+
Courses



151,956+
Sites



236+
Countries

Objectives

1. Deploy Moodle LMS on Kubernetes

- High concurrency handling
- Auto-scaling capability
- High availability & fault tolerance
- **Scope:** Multi-cloud

2. Build a complete CI/CD pipeline

- Automated build, test, and deployment

3. Integrate an AI subsystem

- Local AI (Ollama) or Cloud AI (OpenAI, Gemini, etc.)
to assist learning

Moodle LMS



Open-source learning management system

Pros

- Highly flexible and customizable
- Large, active development community
- Extensive support for various plugins and themes

Target Version: Moodle 5.x

Tech Stack: PHP 8.2-FPM, PostgreSQL 15, Redis

2

Analysis, Design & Implementation

Container orchestration platform

Comparison table

Criteria	Cloud Managed Services	Kubernetes
Vendor Lock-in	High Cloud-dependent, Costly to migrate	None Cloud-agnostic, On-prem ready
Operational Overhead	Minimal Managed by provider	High Requires skilled engineers
Customizability	Limited by provider	Unlimited Highly extensible
Time-to-Market	Rapid Ideal for immediate deployment	Slow initial setup Requires upfront design

Infrastructure cost analysis

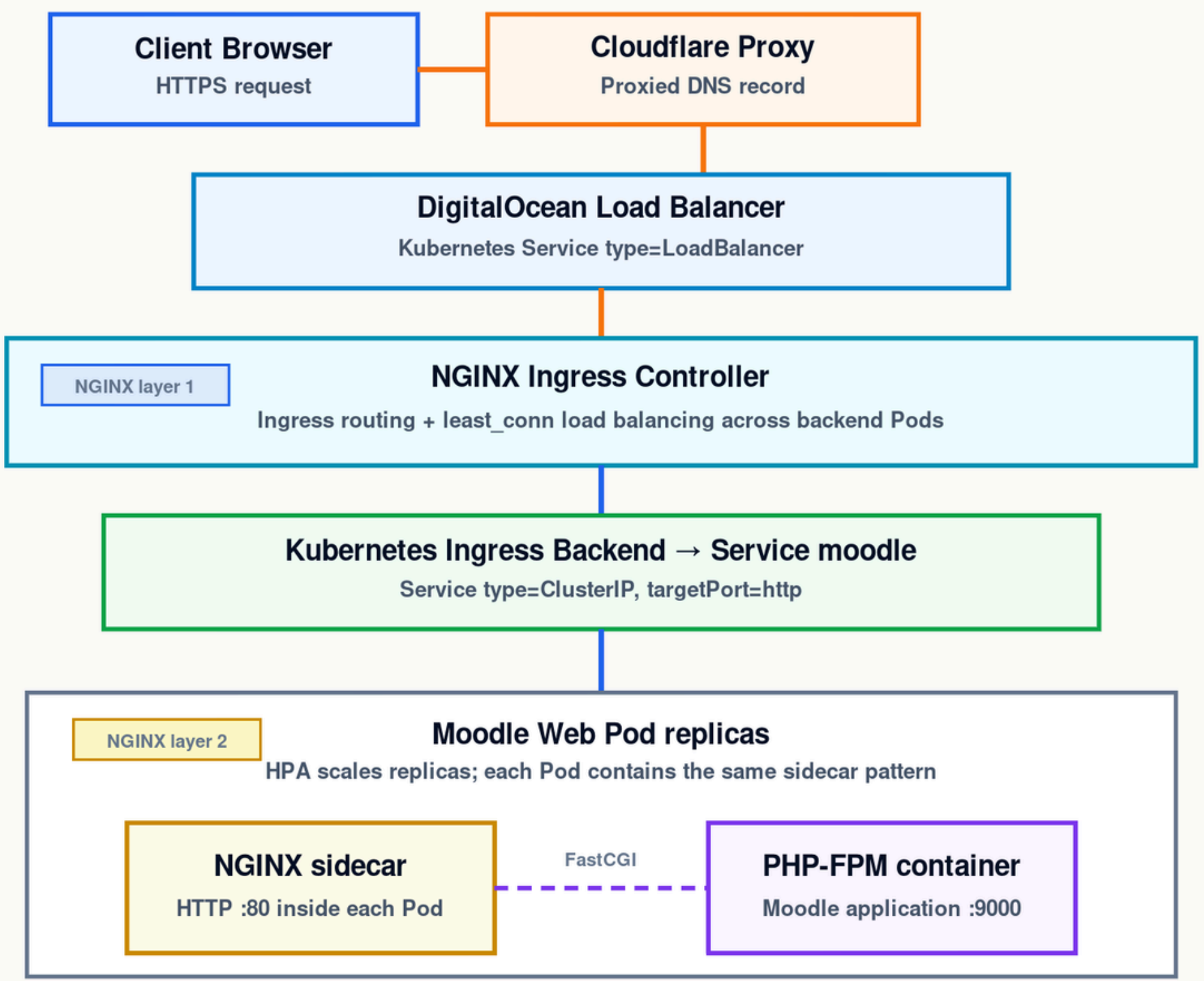
General configuration

- Autoscale 3-6 Node
(Each node 4VCPu, 8GB RAM)
- 2 Databases designed using
Primary + Standby.
(Each DB 2 VCPu, 4GB RAM,
max 97 connection pools))

(Cost/month)	DigitalOcean (DO)	AWS	Azure
K8s Basic	0\$	73\$	0\$
Node Scale 3→6	48\$ 144\$-288\$	~124\$ 372\$-744\$	~124\$ 370\$-740\$
Database	\$122	~\$150	~\$140
Load balancer	\$15.00	~\$30.00	~\$20.00
Tổng	~281 - 425\$	~625 - 997\$	~530 - 900\$

Web Server Architecture Comparison

Comparison table of Nginx vs. Apache



Criteria	Nginx Reverse Proxy & PHP-FPM	Apache Web Server (Monolithic)
I/O Mechanism	Event-driven Non-blocking	Process-based Blocking
Connection Management	1 worker multiple connections	1 process 1 connection
Processing	Decoupled (Static/Dynamic split)	Mixed (One process handles all)
Resource Use	Optimized (Stable RAM)	High (Linear RAM growth)
Scalability	Easy Independent scaling	Difficult High bottleneck risk

Package Manager

Helm is the standard package manager for k8s, helping to package all the complex resources of an application into a single implementation unit called a Chart.

Criteria	Manual YAML	Helm
Resource Management	Decentralized	Centralized
Reusability	Low (Recreated for each environment)	Very high (Shared templates)
Deployment Operation	Run commands per file/folder	Single-command deployment
Version Control	Difficult (Manual tracking)	Built-in revision management & rollback
Suitability	Simple architecture	Complex

Database

- Master - Replica (Write - Read)
 - **Session** mode instead of **transaction** mode
 - Deploy **PgBouncer** for connection stability
 - Managed Database (default):
 - Monitors and alerts admins via email when disk space is full.
 - **No** Auto-scaling storage
- (Relies entirely on the cloud provider).

PostgreSQL

**moodle-db-postgres-production-87644**
in [first-project](#) / 4 GB RAM / 2vCPU / 60 GiB Disk / Primary + Standby / SGP1 - PostgreSQL 16

[Upsize database cluster](#) [Actions](#)

[Overview](#) [Insights](#) [Logs & Queries](#) [Users & Databases](#) **[Connection Pools](#)** [Network Access](#) [Settings](#)

Connection Pools

[Learn](#) [Create a Connection Pool](#)

Available backend server connections: **12 / 97** [?](#)

Pool Name	Database	Username	Pool Mode	Size	Connection details
moodle-pool-production	moodle	moodleuser	session	85	Connection details ...

Settings

[Learn](#)

Project

 first-project [Edit](#)

Configuration

 **Compute** - Basic - Shared CPU / 2 vCPU / 4 GB RAM / Connection limit: 97 [Edit](#)

 **Storage** - 60 GiB SSD

 **Standby** - One additional node

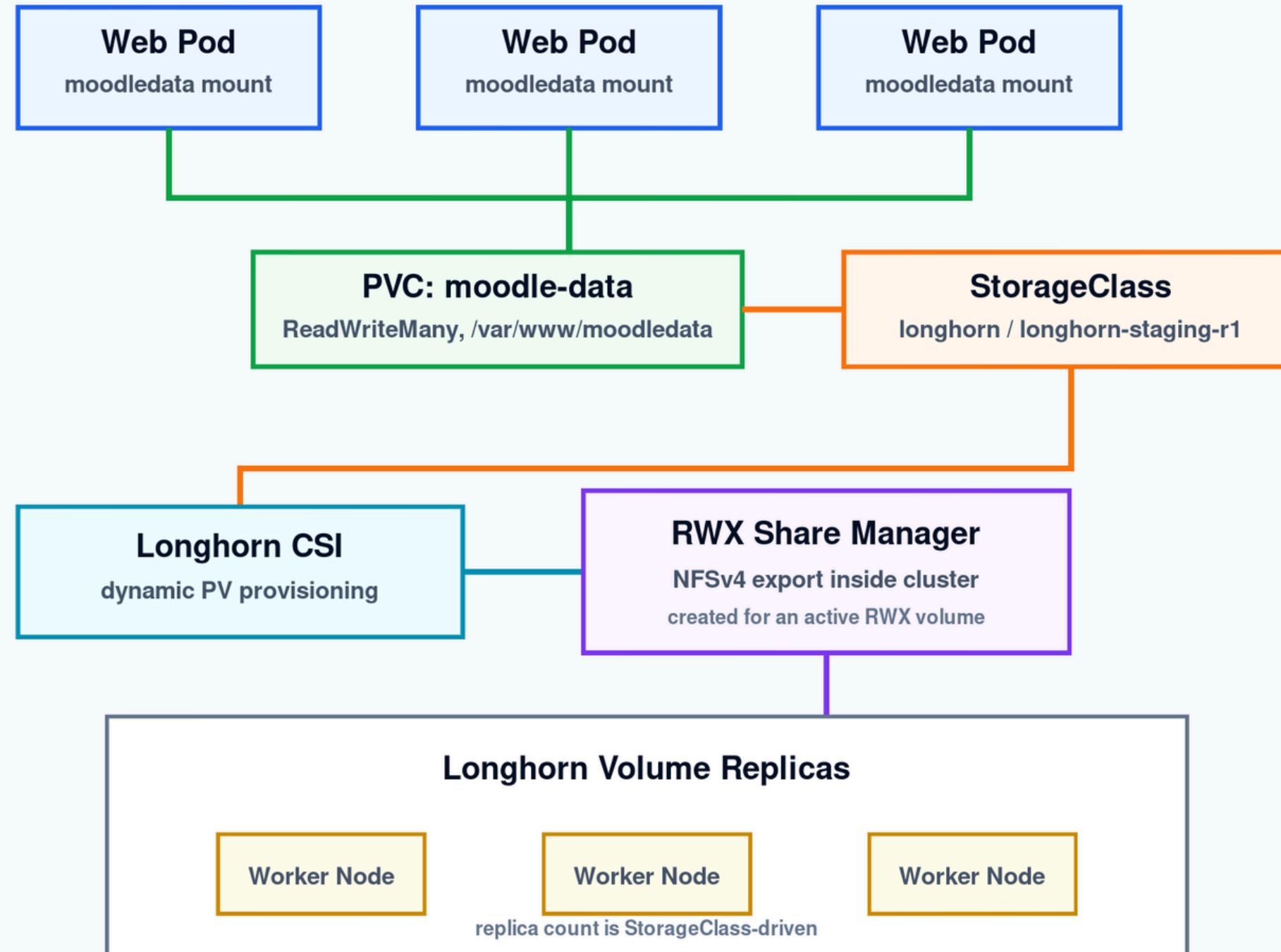
 **Total Cost** - \$121.80/month

Cluster datacenter

 **Singapore** • Datacenter 1 • SGP1 [Edit](#)
VPC network - [default-sgp1](#)

Config & Storage Resources

SHARED FILE STORAGE PATH



RESOURCE PLACEMENT

Longhorn RWX PVC

- Shared Moodle files and uploads
- Mounted at `/var/www/moodledata`

Pod-local runtime

- Moodle app copy on `emptyDir`
- `/tmp/moodlelocalcache`
- OPcache and APCu in memory

ConfigMap / Secret

- ConfigMap: NGINX and cache setup
- Secret: database password

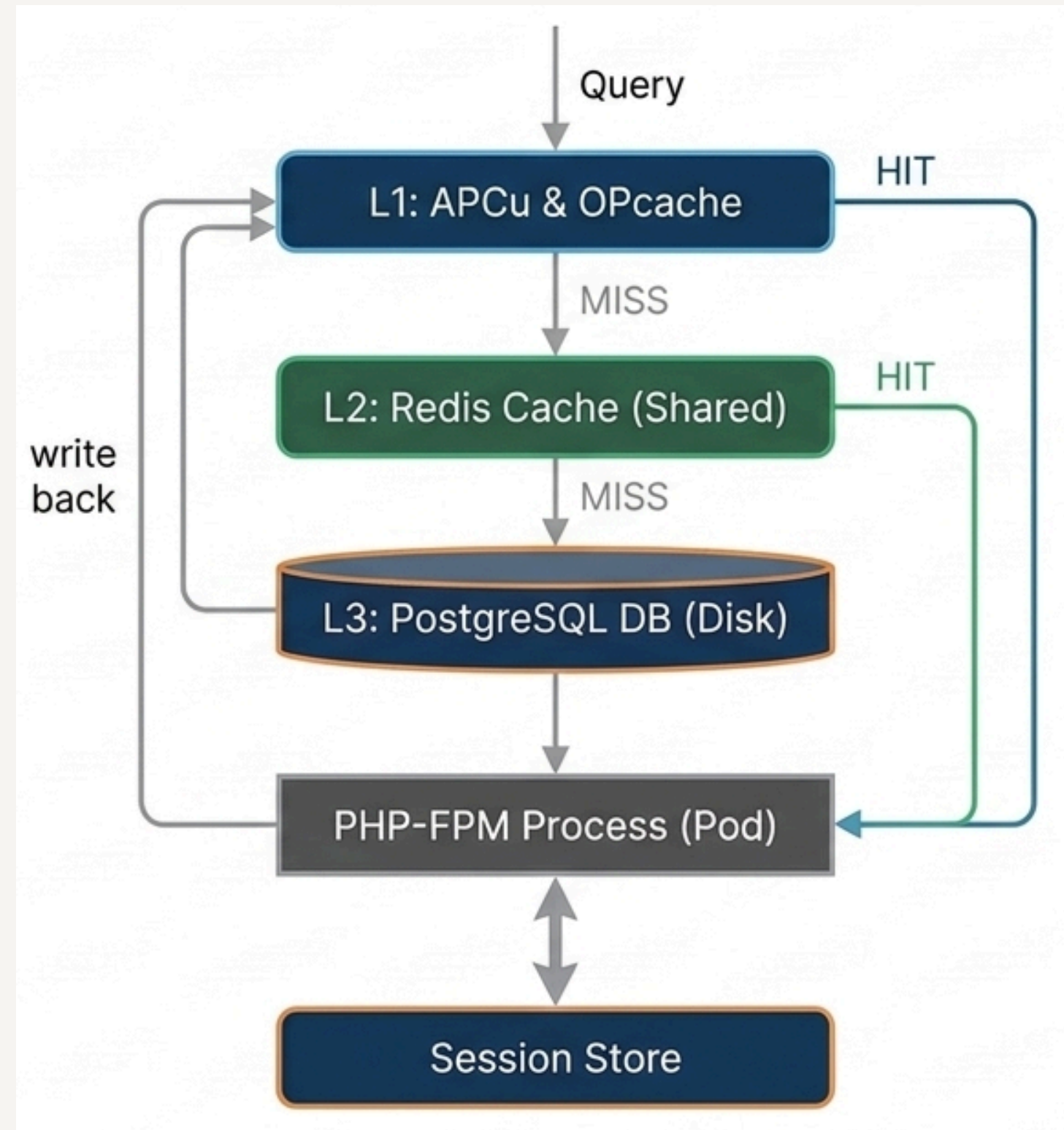
Database is separate

- DigitalOcean Managed PostgreSQL

Longhorn stores shared files, not PostgreSQL data.

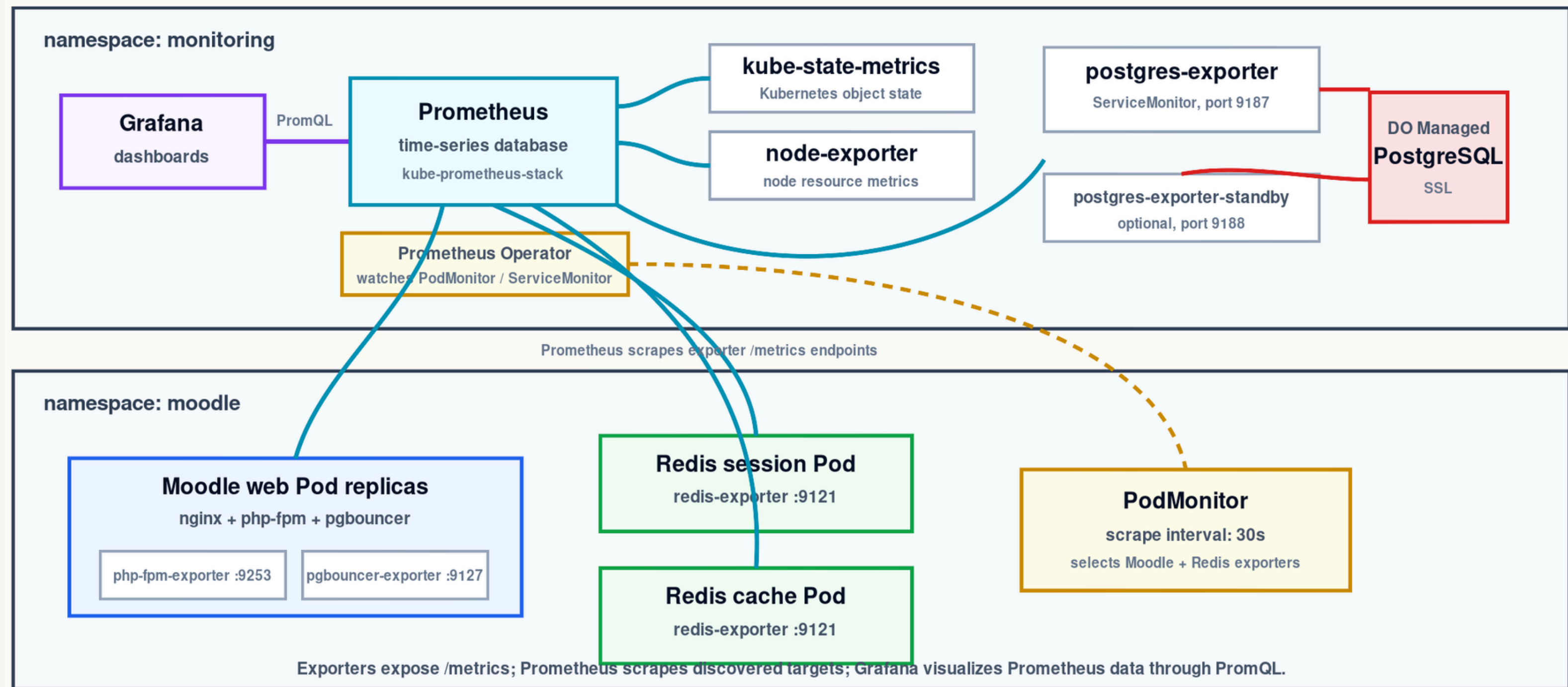
Caching

- **L1** Cache (Per-pod, RAM):
 - **APCu** cache
 - **OPcache** cache
- **L2** (Shared across nodes, RAM):
 - **Redis** cache
- **L3** Storage (disk):
 - **Database**
- **Session Store:** Stores user login sessions.



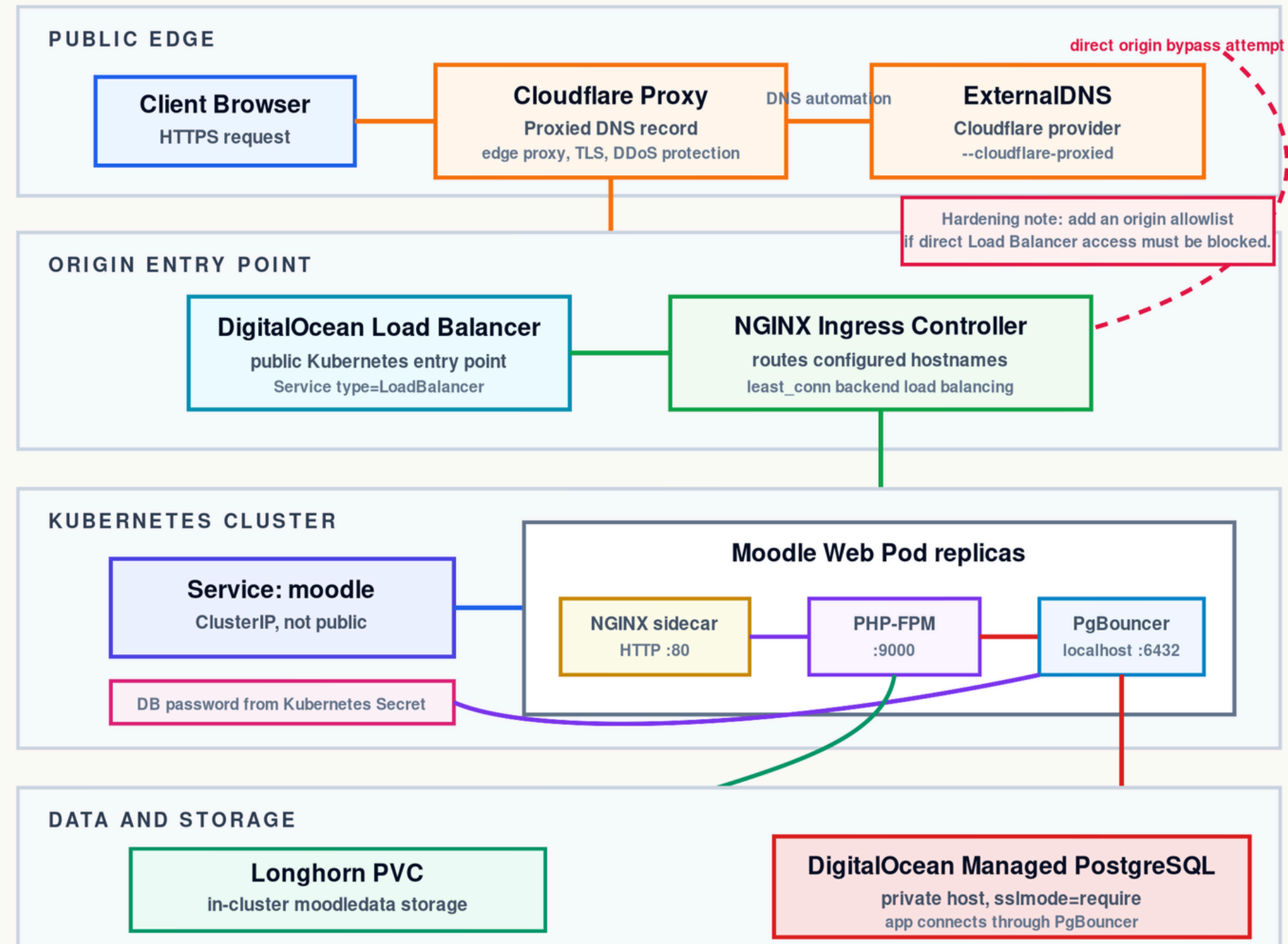
Observability & Metrics

- Separate Grafana into purpose-driven dashboards
- Exporters deployed per pod to collect metrics



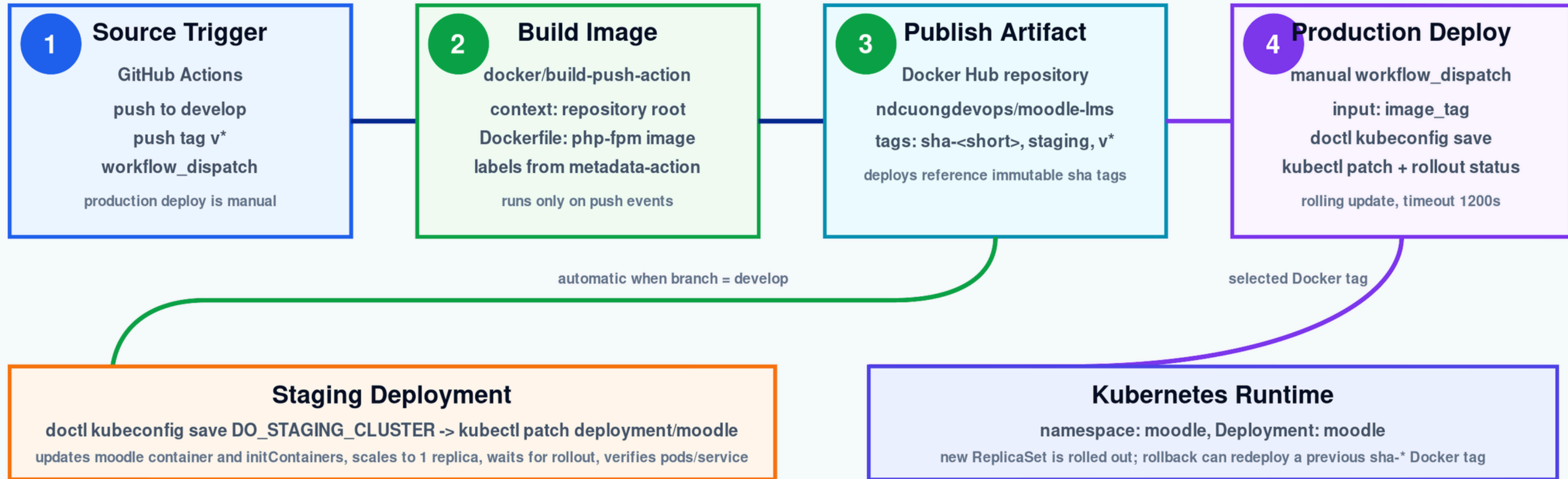
Security

- Infrastructure-layer attacks handled mostly by Cloudflare
- Moodle restricted to domain-only access, blocking IP
- Database restricted strictly to internal Kubernetes access



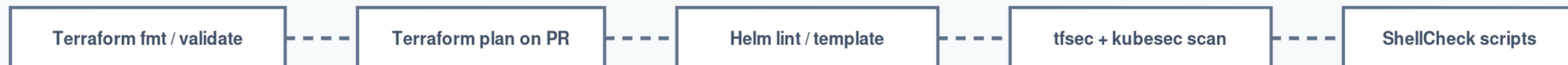
Implementing CI/CD

APPLICATION DELIVERY: moodle-k8s-project / .github/workflows/cicd.yml



Provisioning is separate: digitalocean/setup.sh creates DOKS, Managed PostgreSQL, Ingress, Helm release; CI/CD updates the Moodle image afterward.

INFRASTRUCTURE VALIDATION: moodle-k8s-infra / .github/workflows/validate.yml

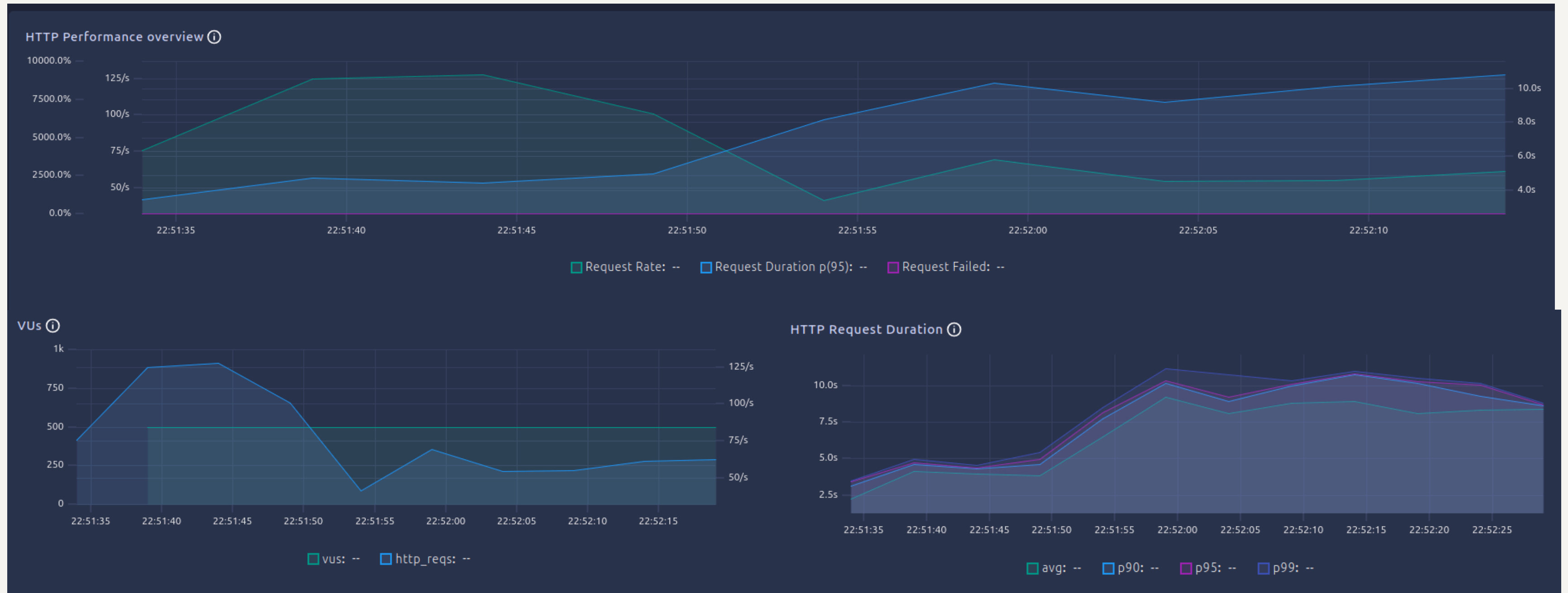




Results and future development

Load capacity

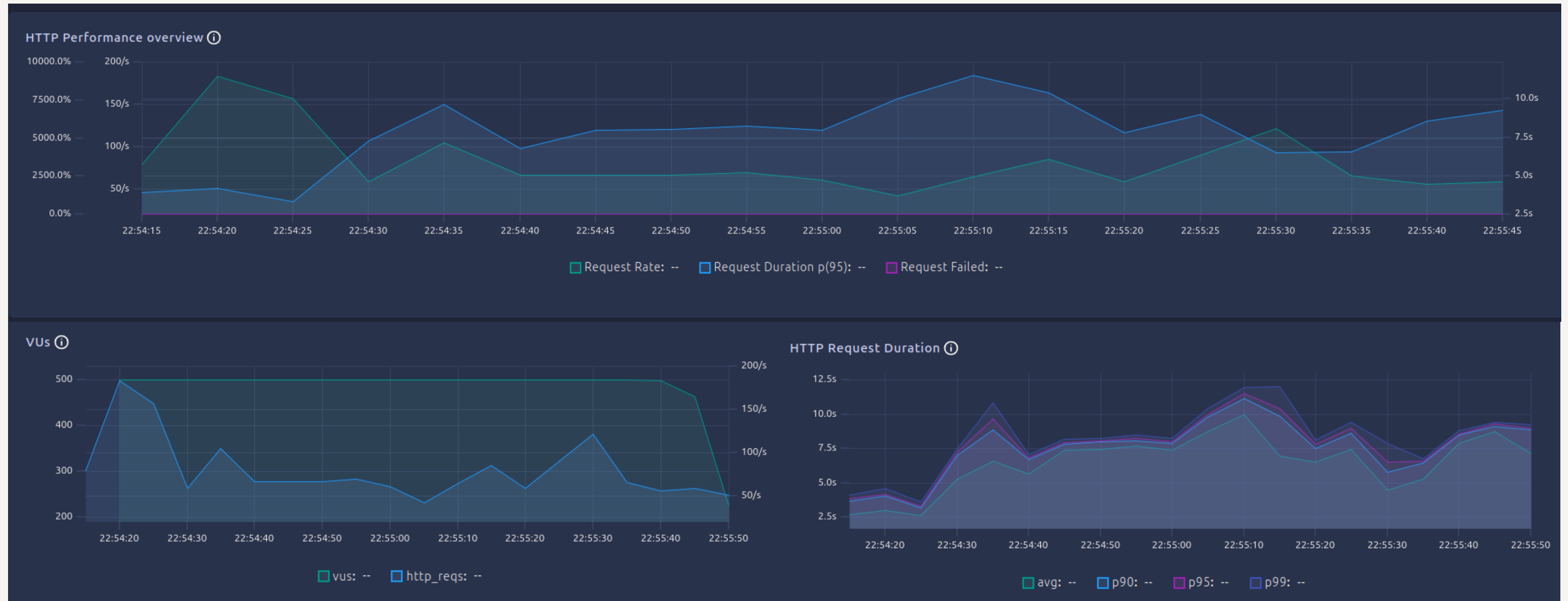
Scenario: 500 users access quiz, then simultaneously submit after 30s



First 30s: Users access quiz and pre-fill answers

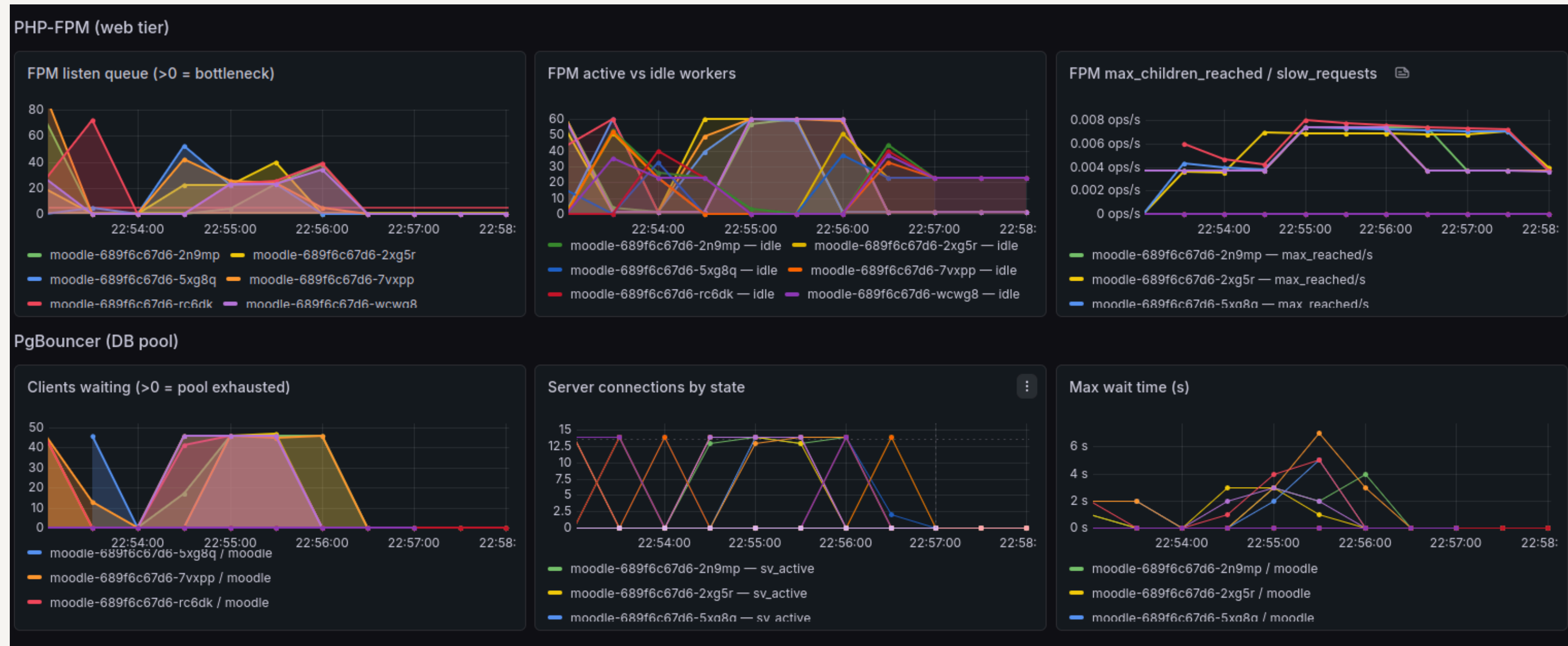
Load capacity

Scenario: 500 users access quiz, then simultaneously submit after 30s



After 30s: Users simultaneously submit quiz at the same time

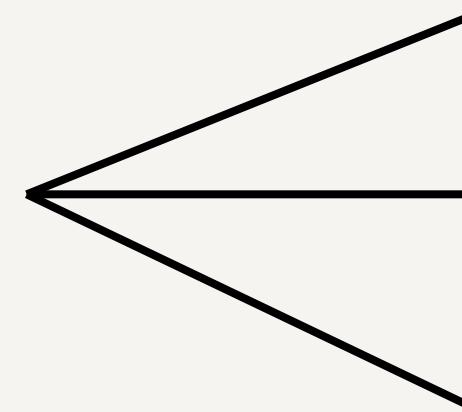
Dashboard Grafana



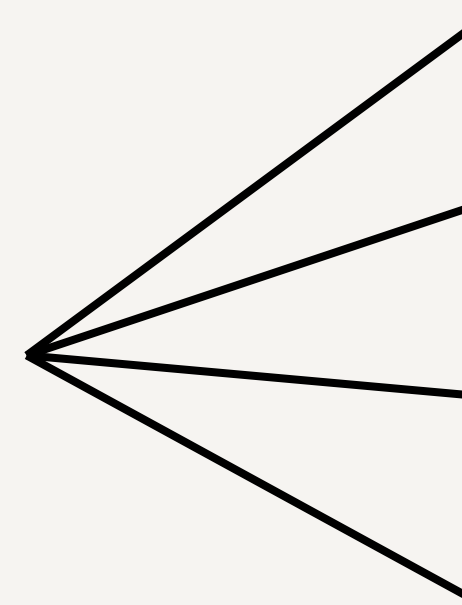
Conclusion:

- Database bottleneck with high PgBouncer queue
- Blocked queries causing App-layer connection backlog

Constraints

- 
- Only short-term testing conducted
 - CI/CD limited to app and infrastructure
 - High infrastructure cost for Moodle AI

Future Development Plans

- 
- Enhance monitoring, alerting, and incident response
 - Implement central logging for daily operations
 - Deploy Cloudflare Waiting Room for downtime
 - Optimize software layer over infrastructure layer

4

DEMO



**Thank You For
Your Listening**